

Algorithms

Dr. Mudassir Shabbir

LUMS University

March 30, 2026



Announcements

- HW 5 is due on [Apr 05](#) at **11:59 AM**.
- **LQ 2** is on [Apr 05](#) at **6:30 PM**.



Example: Edit Distance

Problem: Given two strings A and B , find the **minimum number of operations** to transform A into B .



Example: Edit Distance

Problem: Given two strings A and B , find the **minimum number of operations** to transform A into B .

Allowed operations (each costs 1):

- **Insert** a character.
- **Delete** a character.
- **Replace** a character with another.



Example: Edit Distance

Problem: Given two strings A and B , find the **minimum number of operations** to transform A into B .

Allowed operations (each costs 1):

- **Insert** a character.
- **Delete** a character.
- **Replace** a character with another.

Example: Transform `kitten` $\rightarrow$ `sitting`.

- 1 `kitten` $\rightarrow$ `sitten` (replace `k` $\rightarrow$ `s`)
- 2 `sitten` $\rightarrow$ `sittin` (replace `e` $\rightarrow$ `i`)
- 3 `sittin` $\rightarrow$ `sitting` (insert `g`)

Edit distance = **3**.



Edit Distance: Recurrence

Let $ED(i, j)$ = edit distance between $A[1..i]$ and $B[1..j]$.



Edit Distance: Recurrence

Let $ED(i, j)$ = edit distance between $A[1..i]$ and $B[1..j]$.

Base cases:

$$ED(i, 0) = i, \quad ED(0, j) = j.$$

(Transform prefix of length i to empty string: i deletions, and vice versa.)



Edit Distance: Recurrence

Let $ED(i, j)$ = edit distance between $A[1..i]$ and $B[1..j]$.

Base cases:

$$ED(i, 0) = i, \quad ED(0, j) = j.$$

(Transform prefix of length i to empty string: i deletions, and vice versa.)

Recurrence:

$$ED(i, j) = \begin{cases} ED(i-1, j-1) & \text{if } A[i] = B[j] \text{ (no cost)} \\ 1 + \min \begin{cases} ED(i-1, j) & \text{(delete from } A) \\ ED(i, j-1) & \text{(insert into } A) \\ ED(i-1, j-1) & \text{(replace)} \end{cases} & \text{otherwise} \end{cases}$$



Edit Distance: Recurrence

Let $ED(i, j)$ = edit distance between $A[1..i]$ and $B[1..j]$.

Base cases:

$$ED(i, 0) = i, \quad ED(0, j) = j.$$

(Transform prefix of length i to empty string: i deletions, and vice versa.)

Recurrence:

$$ED(i, j) = \begin{cases} ED(i-1, j-1) & \text{if } A[i] = B[j] \text{ (no cost)} \\ 1 + \min \begin{cases} ED(i-1, j) & \text{(delete from } A) \\ ED(i, j-1) & \text{(insert into } A) \\ ED(i-1, j-1) & \text{(replace)} \end{cases} & \text{otherwise} \end{cases}$$

Answer: $ED(m, n)$ where $m = |A|$, $n = |B|$.



Edit Distance: DP Table

Example: $A = \text{care}$ (length 4), $B = \text{score}$ (length 5). ED = ?

	ϵ	s	c	o	r	e
ϵ						
c						
a						
r						
e						



Edit Distance: DP Table

Example: $A = \text{care}$ (length 4), $B = \text{score}$ (length 5). ED = ?

	ϵ	s	c	o	r	e
ϵ	0	1	2	3	4	5
c						
a						
r						
e						



Edit Distance: DP Table

Example: $A = \text{care}$ (length 4), $B = \text{score}$ (length 5). ED = ?

	ϵ	s	c	o	r	e
ϵ	0	1	2	3	4	5
c	1					
a	2					
r	3					
e	4					



Edit Distance: DP Table

Example: $A = \text{care}$ (length 4), $B = \text{score}$ (length 5). ED = ?

	ϵ	s	c	o	r	e
ϵ	0	1	2	3	4	5
c	1	1	1	2	3	4
a	2					
r	3					
e	4					



Edit Distance: DP Table

Example: $A = \text{care}$ (length 4), $B = \text{score}$ (length 5). ED = ?

	ϵ	s	c	o	r	e
ϵ	0	1	2	3	4	5
c	1	1	1	2	3	4
a	2	2	2	2	3	4
r	3					
e	4					



Edit Distance: DP Table

Example: $A = \text{care}$ (length 4), $B = \text{score}$ (length 5). ED = ?

	ϵ	s	c	o	r	e
ϵ	0	1	2	3	4	5
c	1	1	1	2	3	4
a	2	2	2	2	3	4
r	3	3	3	3	2	3
e	4					



Edit Distance: DP Table

Example: $A = \text{care}$ (length 4), $B = \text{score}$ (length 5). ED = ?

	ϵ	s	c	o	r	e
ϵ	0	1	2	3	4	5
c	1	1	1	2	3	4
a	2	2	2	2	3	4
r	3	3	3	3	2	3
e	4	4	4	4	3	2



Edit Distance: DP Table

Example: $A = \text{care}$ (length 4), $B = \text{score}$ (length 5). $\text{ED} = ?$

	ϵ	s	c	o	r	e
ϵ	0	1	2	3	4	5
c	1	1	1	2	3	4
a	2	2	2	2	3	4
r	3	3	3	3	2	3
e	4	4	4	4	3	2

Edit distance = $\text{ED}(4, 5) = \mathbf{2}$ (insert s at front; replace a $\rightarrow$ o).



Edit Distance: DP Table

Example: $A = \text{care}$ (length 4), $B = \text{score}$ (length 5). $\text{ED} = ?$

	ϵ	s	c	o	r	e
ϵ	0	1	2	3	4	5
c	1	1	1	2	3	4
a	2	2	2	2	3	4
r	3	3	3	3	2	3
e	4	4	4	4	3	2

Edit distance = $\text{ED}(4, 5) = \mathbf{2}$ (insert s at front; replace a $\rightarrow$ o).

Table structure

Each cell $[i][j]$ depends only on $[i-1][j]$, $[i][j-1]$, and $[i-1][j-1]$ — the same three-neighbor pattern as LCS.

Edit Distance: Pseudocode & Complexity

```
procedure EditDistance( $A$ ,  $B$ )  
   $m \leftarrow |A|$ ;  $n \leftarrow |B|$   
  for  $i = 0$  to  $m$  do  $ED[i][0] \leftarrow i$   
  for  $j = 0$  to  $n$  do  $ED[0][j] \leftarrow j$   
  for  $i = 1$  to  $m$  do  
    for  $j = 1$  to  $n$  do  
      if  $A[i] = B[j]$  then  
         $ED[i][j] \leftarrow ED[i-1][j-1]$   
      else  
         $ED[i][j] \leftarrow 1 + \min(ED[i-1][j], ED[i][j-1], ED[i-1][j-1])$   
  return  $ED[m][n]$ 
```



Edit Distance: Pseudocode & Complexity

```
procedure EditDistance( $A$ ,  $B$ )  
   $m \leftarrow |A|$ ;  $n \leftarrow |B|$   
  for  $i = 0$  to  $m$  do  $ED[i][0] \leftarrow i$   
  for  $j = 0$  to  $n$  do  $ED[0][j] \leftarrow j$   
  for  $i = 1$  to  $m$  do  
    for  $j = 1$  to  $n$  do  
      if  $A[i] = B[j]$  then  
         $ED[i][j] \leftarrow ED[i-1][j-1]$   
      else  
         $ED[i][j] \leftarrow 1 + \min(ED[i-1][j], ED[i][j-1], ED[i-1][j-1])$   
  return  $ED[m][n]$ 
```

Time: $O(m \times n)$. **Space:** $O(m \times n)$; reducible to $O(\min(m, n))$.



Edit Distance: Pseudocode & Complexity

```
procedure EditDistance( $A, B$ )  
   $m \leftarrow |A|$ ;  $n \leftarrow |B|$   
  for  $i = 0$  to  $m$  do  $ED[i][0] \leftarrow i$   
  for  $j = 0$  to  $n$  do  $ED[0][j] \leftarrow j$   
  for  $i = 1$  to  $m$  do  
    for  $j = 1$  to  $n$  do  
      if  $A[i] = B[j]$  then  
         $ED[i][j] \leftarrow ED[i-1][j-1]$   
      else  
         $ED[i][j] \leftarrow 1 + \min(ED[i-1][j], ED[i][j-1], ED[i-1][j-1])$   
  return  $ED[m][n]$ 
```

Time: $O(m \times n)$. **Space:** $O(m \times n)$; reducible to $O(\min(m, n))$.

Applications: spell checking, DNA sequence alignment, diff tools, NLP (word similarity).



Knapsack: A Packing Problem

Bag capacity: $W = 6$ kg. **Goal:** choose items to **maximize total value**.

Available items

i_1 : 1 kg, \$2

i_2 : 3 kg, \$5

i_3 : 2 kg, \$3

i_4 : 4 kg, \$7

i_5 : 2 kg, \$4

Backpack

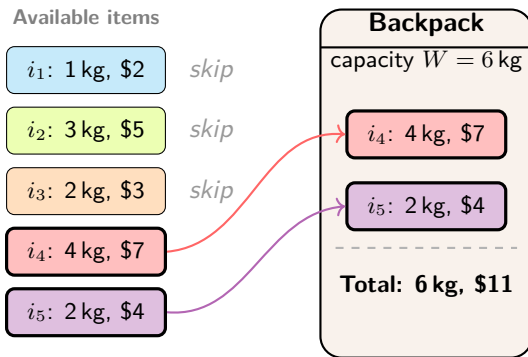
capacity $W = 6$ kg

?



Knapsack: A Packing Problem

Bag capacity: $W = 6$ kg. **Goal:** choose items to **maximize total value**.



Brute force tries all 2^n subsets — too slow. We need a smarter approach: **dynamic programming**.



Example: Knapsack Problem

0/1 Knapsack: Given items with weights and values, select items to maximize value without exceeding capacity W .



Example: Knapsack Problem

0/1 Knapsack: Given items with weights and values, select items to maximize value without exceeding capacity W .

Optimal Substructure:

$$KS(i, w) = \max \begin{cases} KS(i-1, w) & \text{(skip item } i) \\ \text{value}_i + KS(i-1, w - \text{weight}_i) & \text{(include item } i) \end{cases}$$



Example: Knapsack Problem

0/1 Knapsack: Given items with weights and values, select items to maximize value without exceeding capacity W .

Optimal Substructure:

$$KS(i, w) = \max \begin{cases} KS(i-1, w) & \text{(skip item } i) \\ \text{value}_i + KS(i-1, w - \text{weight}_i) & \text{(include item } i) \end{cases}$$

DP table: rows = items ($0 \dots n$), columns = capacities ($0 \dots W$).



Example: Knapsack Problem

0/1 Knapsack: Given items with weights and values, select items to maximize value without exceeding capacity W .

Optimal Substructure:

$$KS(i, w) = \max \begin{cases} KS(i-1, w) & \text{(skip item } i) \\ \text{value}_i + KS(i-1, w - \text{weight}_i) & \text{(include item } i) \end{cases}$$

DP table: rows = items ($0 \dots n$), columns = capacities ($0 \dots W$).

Time: $O(n \times W)$ (pseudo-polynomial in W).

Space: $O(n \times W)$ fully; can reduce to $O(W)$ with rolling array.



Example: Knapsack Problem

0/1 Knapsack: Given items with weights and values, select items to maximize value without exceeding capacity W .

Optimal Substructure:

$$KS(i, w) = \max \begin{cases} KS(i-1, w) & \text{(skip item } i) \\ \text{value}_i + KS(i-1, w - \text{weight}_i) & \text{(include item } i) \end{cases}$$

DP table: rows = items ($0 \dots n$), columns = capacities ($0 \dots W$).

Time: $O(n \times W)$ (pseudo-polynomial in W).

Space: $O(n \times W)$ fully; can reduce to $O(W)$ with rolling array.

Note

If weights are real/unbounded, KS becomes NP-hard. DP only works when capacity is integer-bounded.

Knapsack: DP Table

Items (weight, value): $i_1 = (1, 2)$, $i_2 = (3, 5)$, $i_3 = (2, 3)$, $i_4 = (4, 7)$, $i_5 = (2, 4)$. $W = 6$.

$KS(i, w) = \text{max value using items } 1 \dots i \text{ with capacity } w$.

	0	1	2	3	4	5	6
$\emptyset$							
i_1							
i_2							
i_3							
i_4							
i_5							



Knapsack: DP Table

Items (weight, value): $i_1 = (1, 2)$, $i_2 = (3, 5)$, $i_3 = (2, 3)$, $i_4 = (4, 7)$, $i_5 = (2, 4)$. $W = 6$.

$KS(i, w) = \text{max value using items } 1 \dots i \text{ with capacity } w$.

	0	1	2	3	4	5	6
$\emptyset$	0	0	0	0	0	0	0
i_1							
i_2							
i_3							
i_4							
i_5							



Knapsack: DP Table

Items (weight, value): $i_1 = (1, 2)$, $i_2 = (3, 5)$, $i_3 = (2, 3)$, $i_4 = (4, 7)$, $i_5 = (2, 4)$. $W = 6$.

$KS(i, w) = \text{max value using items } 1 \dots i \text{ with capacity } w$.

	0	1	2	3	4	5	6
$\emptyset$	0	0	0	0	0	0	0
i_1	0						
i_2	0						
i_3	0						
i_4	0						
i_5	0						



Knapsack: DP Table

Items (weight, value): $i_1 = (1, 2)$, $i_2 = (3, 5)$, $i_3 = (2, 3)$, $i_4 = (4, 7)$, $i_5 = (2, 4)$. $W = 6$.

$KS(i, w) = \max$ value using items $1 \dots i$ with capacity w .

	0	1	2	3	4	5	6
$\emptyset$	0	0	0	0	0	0	0
i_1	0	2	2	2	2	2	2
i_2	0						
i_3	0						
i_4	0						
i_5	0						



Knapsack: DP Table

Items (weight, value): $i_1 = (1, 2)$, $i_2 = (3, 5)$, $i_3 = (2, 3)$, $i_4 = (4, 7)$, $i_5 = (2, 4)$. $W = 6$.

$KS(i, w) = \text{max value using items } 1 \dots i \text{ with capacity } w$.

	0	1	2	3	4	5	6
$\emptyset$	0	0	0	0	0	0	0
i_1	0	2	2	2	2	2	2
i_2	0	2	2	5	7	7	7
i_3	0						
i_4	0						
i_5	0						



Knapsack: DP Table

Items (weight, value): $i_1 = (1, 2)$, $i_2 = (3, 5)$, $i_3 = (2, 3)$, $i_4 = (4, 7)$, $i_5 = (2, 4)$. $W = 6$.

$KS(i, w) = \max$ value using items $1 \dots i$ with capacity w .

	0	1	2	3	4	5	6
$\emptyset$	0	0	0	0	0	0	0
i_1	0	2	2	2	2	2	2
i_2	0	2	2	5	7	7	7
i_3	0	2	3	5	7	8	10
i_4	0						
i_5	0						



Knapsack: DP Table

Items (weight, value): $i_1 = (1, 2)$, $i_2 = (3, 5)$, $i_3 = (2, 3)$, $i_4 = (4, 7)$, $i_5 = (2, 4)$. $W = 6$.

$KS(i, w) = \max$ value using items $1 \dots i$ with capacity w .

	0	1	2	3	4	5	6
$\emptyset$	0	0	0	0	0	0	0
i_1	0	2	2	2	2	2	2
i_2	0	2	2	5	7	7	7
i_3	0	2	3	5	7	8	10
i_4	0	2	3	5	7	9	10
i_5	0						



Knapsack: DP Table

Items (weight, value): $i_1 = (1, 2)$, $i_2 = (3, 5)$, $i_3 = (2, 3)$, $i_4 = (4, 7)$, $i_5 = (2, 4)$. $W = 6$.

$KS(i, w) = \max$ value using items $1 \dots i$ with capacity w .

	0	1	2	3	4	5	6
$\emptyset$	0	0	0	0	0	0	0
i_1	0	2	2	2	2	2	2
i_2	0	2	2	5	7	7	7
i_3	0	2	3	5	7	8	10
i_4	0	2	3	5	7	9	10
i_5	0	2	4	6	7	9	11



Knapsack: DP Table

Items (weight, value): $i_1 = (1, 2)$, $i_2 = (3, 5)$, $i_3 = (2, 3)$, $i_4 = (4, 7)$, $i_5 = (2, 4)$. $W = 6$.

$KS(i, w) = \text{max value using items } 1 \dots i \text{ with capacity } w$.

	0	1	2	3	4	5	6
$\emptyset$	0	0	0	0	0	0	0
i_1	0	2	2	2	2	2	2
i_2	0	2	2	5	7	7	7
i_3	0	2	3	5	7	8	10
i_4	0	2	3	5	7	9	10
i_5	0	2	4	6	7	9	11

Max value = $KS(5, 6) = 11$ (take i_4 and i_5 : weight $4 + 2 = 6$, value $7 + 4 = 11$).

